Simulation Design and Modeling Strategy

The Influence of Reviewer Expertise and Engagement on Peer Review of Grants

Sam Harper & Arijit Nandi

2026-09-02

1 Overview

This is a design and validation exercise for a project that aims to assess how reviewer engagement and expertise may affect post-discussion grant scores during CIHR peer review. Given the restrictive nature of CIHR funding data, this document lays out the data-generating process we believe is similar to CIHR's Project Grant peer review, the model we intend to fit against it, and evidence that the models we will use can actually recover simulated effects.

It is not a full pre-analysis plan (no pre-specified hypotheses), but it's close in spirit, and a subsequent revision could become one.

1.1 Research aims

1. **Aim 1** – How do reviewer expertise (self-described: high/medium/low/not enough) and engagement (assigned reviewer vs. non-reviewing panelist) affect the impact of panel discussion on scores?
2. **Aim 2** – Do these effects differ by applicant characteristics, namely gender or career stage?
3. **Aim 3** (exploratory) – How would alternative funding-decision schemes (e.g. reweighting by engagement, partial randomization near the funding threshold) compare to the status quo?

1.2 Project Scheme review process

- The 3 assigned reviewers read the application and score it. At the meeting (and *before* any discussion), they are required to agree on a **consensus score**.
- After discussion all panel members (including the 3 reviewers) submit a **final score**. Reviewers are not bound to their own consensus number; they can move too.
- The final score must fall within **±0.5** of the consensus score, and is entered to **one decimal place**.
- The (equally weighted) average of final scores across all panel members feeds the funding decision.

1.3 Data structure

We're assuming a three-level structure: CIHR-established committees, applications nested in committees, and panel members nested in committees (crossed with applications, since each member reviews many applications within a cycle). Based on recent Project Grant committee sizes (cite webpage?), our working numbers are roughly 50 committees, but we allow the number of applications across committees to vary based on CIHR documentation, and simulate the number of applications proportionally (with some variance)

CIHR's Funding Analytics Team confirmed by email (2025-12-04) which fields are actually extractable for a data pull, versus fields that can only ever be touched by a CIHR analyst running our code in-house on the real data (full table and follow-up questions in `code/ror-research-log.qmd`). Headline for this document: committee/application/member identifiers, role (reviewer vs. panelist), self-described expertise, initial reviewer scores, consensus score, final scores, and funding result are all extractable. **Applicant gender and career stage – Aim 2′s entire basis – are not**, and won't appear even in CIHR's own distribution-matched dummy data. That constraint is why this document exists: the simulation below is the only rehearsal Aim 2′s code gets before it runs once, unsupervised, on real data.

2 Modeling Strategy

2.1 Primary outcome(s)

The primary Aim 1 outlined above is to assess how *changes* from the initial consensus score to the final review scores (after discussion) may vary with panel expertise and engagement. Thus, rather than our primary outcome being the overall application score, we specifically want to model the difference between the consensus score and the final scores.

Let d_{ijk} be the final score minus the consensus score, for the ith panel member on the jth application in the kth committee:

$$d_{ijk} = \text{final score}_{ijk} - \text{consensus score}_{jk}$$

Our interest is in modeling d_{ijk} directly, rather than modeling the final score with consensus score as a covariate, for three reasons:

1. **Point mass at zero.** We hypothesize that most or many members will go with the consensus score, which leads to a probability mass either at the consensus (if you modeled the score) or a mass at zero if you model the difference. A member who doesn't move from consensus has $d_{ijk} = 0$ regardless of which application they're scoring. So even if you model the final score conditional on the consensus you will still have a spike, just relocated to wherever a given application's consensus happened to land. Differencing standardizes the spike's location (at zero) across every application and makes a shared two-part model more feasible.
2. **Avoids some sources of confounding.** The consensus score is constant across everyone in the committee and reflects both the reviewer's and the application's initial "true quality" signal. Subtracting it off removes anything that's shared by every member evaluating the same application (committee identity, application quality, etc.), leaving only the within-application variation across panel members as what's left to explain. If some committees have both higher consensus scores and a different mix of reviewer expertise then consensus and 'expertise' are correlated across applications. Differencing eliminates this potential bias since the coefficient on consensus isn't estimated at all.
3. **Matches the actual research question.** Aim 1 is about the *change from consensus* induced by discussion, not the *level* of the final score (driven mostly by application quality). Since it is simple to recover the final score like $\text{final score} = \text{consensus} + d_{ijk}$, anything Aim 3 needs at the score level for funding-decision simulations can be reconstructed downstream.

2.2 Estimand of interest

Our main quantity of interest in this project is the value that a given committee member's score differential from the consensus (d_{ijk}) would take if a particular reviewer characteristic (role or experience) were set to a specific value, averaged over the entire population of panel members, applications, and committees. We can write an example of, say, the difference between differential scores for a given member-application pairing if that member were assigned to be a panelist or a reviewer:

$$\underbrace{\frac{1}{IJK}\sum_{k=1}^{K}\sum_{j=1}^{J}\sum_{i=1}^{I}}_{\substack{\text{Mean over members } i,\\ \text{applications } j,\\ \text{and committees } k}}\underbrace{\big(d_{ijk}(1) - d_{ijk}(0)\big)}_{\substack{\text{Score if assigned}\\ \text{to panelist}(1)\\ \text{or reviewer}(0)}}$$

Leaving aside for the moment the assumptions needed to credibly estimate this quantity, the specific data generating process for score deviations leads to challenges, since the distribution of this outcome depends on two processes: 1) whether a given member deviates from the consensus score; and 2) how large that deviation might be. Estimating these effects across the whole population of applications leads to the intuition for a two-part model.

2.3 Two-part model

We expect d_{ijk} to have a spike at exactly zero (members who simply adopt the consensus score) plus variation among those who depart from it. So the outcome is split into two linked models:

1. **Did this member deviate at all?** A binary/logistic model on $1\big[d_{ijk} \neq 0\big]$.
2. **If they deviated, by how much?** A model for the (signed) magnitude, conditional on deviating.

Estimating the expected value of the both parts across the entire population of reviewers (not just those that deviate from consensus) recovers the effect in the entire sample. This is akin to a 'hurdle' type model (Mullahy 1986) that combines a binary outcome and a magnitude estimate for non-zero outcomes.

$$E\big[d_{ijk}\big] = \underbrace{P(\text{deviate})}_{\substack{\text{Part 1: any}\\ \text{deviation at all}}} \times \underbrace{E[\text{magnitude} \mid \text{deviate}]}_{\substack{\text{Part 2: size}\\ \text{given deviation}}}$$

2.4 Model specification (Aim 1)

We adopt a Bayesian modeling approach and model the binomial outcome of whether or not a member deviated as a function of their engagement with the application (reviewer vs. panelist) and their self-declared expertise to review (high, medium, low, none). In our specification below D_{ijk} is 1 for those who deviated, and we include random effects for committees ($\gamma_{cmte[k]}$), members ($\alpha_{mem[i]}$), and applications ($\beta_{app[j]}$). For the fixed effects δ is the effect of being a panelist vs. reviewer, and ζ captures the suite of effects for the expertise indicators ($\mathbf{Exp}_{ij}$)

$$
\begin{aligned}
D_{\text{ijk}} &\sim \text{Binomial}\left(1, p_{\text{ijk}}\right) && \text{[likelihood]} \\
\text{logit}\left(p_{\text{ijk}}\right) &= \alpha_{\text{mem}_i} + \gamma_{\text{cmte}_k} + \beta_{\text{app}_j} + \delta \text{Panelist}_{\text{ijk}} + \zeta \mathbf{Exp}_{\text{ij}} && \text{[linear model for log odds]} \\
\alpha_{\text{mem}_i} &\sim \text{Normal}\left(\bar{\alpha}, \sigma_\alpha\right) && \text{[prior for member intercepts]} \\
\gamma_{\text{cmte}_k} &\sim \text{Normal}\left(0, \sigma_\gamma\right) && \text{[prior for committee intercepts]} \\
\beta_{\text{app}_j} &\sim \text{Normal}\left(0, \sigma_\beta\right) && \text{[prior for application intercepts]} \\
\left(\delta, \zeta\right) &\sim \text{Normal}(0, 0.5) && \text{[prior for fixed effects]} \\
\bar{\alpha} &\sim \text{Normal}(0, 1.0) && \text{[prior for average member]} \\
\left(\sigma_\alpha, \sigma_\gamma, \sigma_\beta\right) &\sim \text{Exponential}(1) && \text{[prior for standard deviations]}
\end{aligned}
$$

Bayesian models require priors on all parameters, and we generally plan to use weakly regularizing priors that allow for a wide range of potential effects but that are generally skeptical of effects of large magnitude. Below we show distributions that helped frame our decision. For the overall probability of deviating the Normal(0, 1.0) prior (green line in top plot) provides a generally wide possibility of the probability of deviating (95% of the probability mass is between 14% and 88% deviating), and a Normal(0, 0.5) for the beta coefficients puts 95% of the probability mass on differences in the probability of deviating of +/- 20 percentage points (i.e., it is quite skeptical of very large effect sizes).

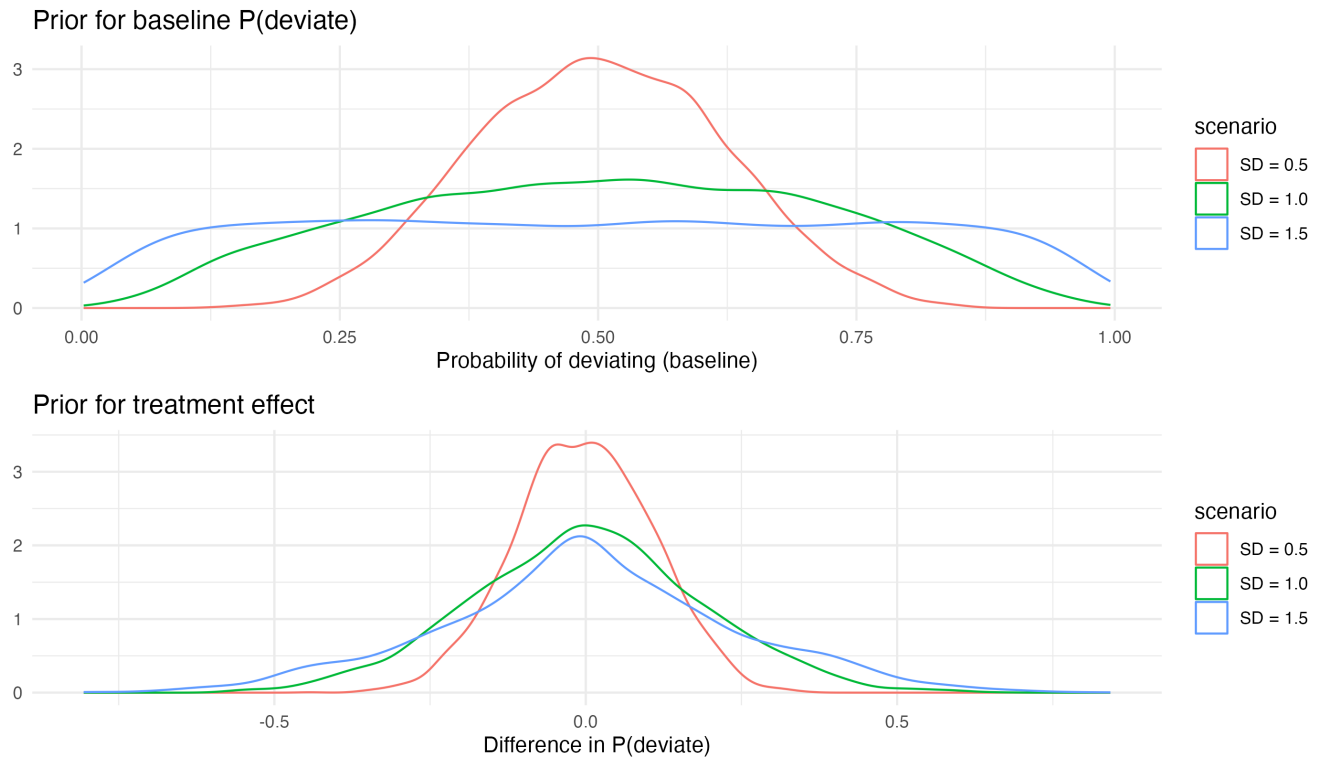

Figure 1: Hypothetical priors for deviation model

2.5 Model specification: magnitude, given a deviation

The second part of the model asks how large a deviation occurred among members who deviated, i.e., M_{ijk} as one of 10 discrete steps ($\pm 0.1 \ldots \pm 0.5$) either positive or negative. Since this outcome has fixed categories and a natural ordering (from –0.5 to + 0.5) we use an ordinal cumulative-logit model with flexible (non-equidistant) thresholds τ_c. One difference from Part 1 is worth flagging: because the

$K - 1 = 9$ thresholds already anchor the model's location, none of the group-level intercepts needs to absorb a population mean the way $\alpha_{mem[i]}$ did above – $\gamma_{cmte[k]}$, $\alpha_{mem[i]}$, and $\beta_{app[j]}$ are all mean-zero here. The other difference is new as of this round of simulation work: role (reviewer vs. panelist) doesn't just shift *where* a member's deviation tends to land (η_{ijk}), it also affects *how spread out* the deviation is (κ_{ijk}, brms's discrimination parameter for ordinal models) – panelists show a genuinely wider distribution of deviation sizes, not just a different average. For consistency with how η_{ijk} is written above, κ_{ijk} is shown here using the same reference-level (Reviewer) plus contrast (Panelist) convention: an intercept δ_0^{disc} and a contrast δ_1^{disc}. Unlike η_{ijk}, though, κ_{ijk} has no random effects or thresholds to implicitly absorb a reference value, so it needs an explicit intercept term of its own – both δ_0^{disc} and δ_1^{disc} are still freely estimated (Reviewer's own discrimination isn't pinned at any fixed value here). The model is actually fit in code as `disc ~ 0 + job` (each role's discrimination estimated directly, rather than as an intercept-plus-contrast) – algebraically equivalent to what's shown below, chosen there for direct interpretability against `brms`'s neutral default of $\kappa = 1$ for everyone (see research log, 2026-08-27, for the full reasoning); the reference-contrast form here is purely to keep this equation's presentation consistent with η_{ijk}'s.

$$
\begin{aligned}
M_{\text{ijk}} \mid D_{\text{ijk}} = 1 &\sim \text{Cumulative}\big(\eta_{\text{ijk}}, \kappa_{\text{ijk}}, \boldsymbol{\tau}\big) && \text{[likelihood, 10 ordered categories]} \\
P\big(M_{\text{ijk}} \le c\big) &= \text{logit}^{-1}\big(\kappa_{\text{ijk}}\big(\tau_c - \eta_{\text{ijk}}\big)\big) && \text{[cumulative-logit link]} \\
\eta_{\text{ijk}} &= \alpha_{\text{mem}_i} + \gamma_{\text{cmte}_k} + \beta_{\text{app}_j} + \delta\text{Panelist}_{\text{ijk}} + \zeta\mathbf{Exp}_{\text{ij}} && \text{[location]} \\
\log\big(\kappa_{\text{ijk}}\big) &= \delta_0^{\text{disc}} + \delta_1^{\text{disc}}\text{Panelist}_{\text{ijk}} && \text{[role-specific discrimination]} \\
\alpha_{\text{mem}_i} &\sim \text{Normal}(0, \sigma_\alpha) && \text{[prior for member intercepts]} \\
\gamma_{\text{cmte}_k} &\sim \text{Normal}\big(0, \sigma_\gamma\big) && \text{[prior for committee intercepts]} \\
\beta_{\text{app}_j} &\sim \text{Normal}\big(0, \sigma_\beta\big) && \text{[prior for application intercepts]} \\
\tau_c &\sim \text{Normal}(0, 1.5), c = 1, ..., 9 && \text{[prior for thresholds]} \\
(\delta, \zeta) &\sim \text{Normal}(0, 0.5) && \text{[prior for location fixed effects]} \\
\big(\delta_0^{\text{disc}}, \delta_1^{\text{disc}}\big) &\sim \text{Normal}(0, 1) && \text{[prior for discrimination effects]} \\
\big(\sigma_\alpha, \sigma_\gamma, \sigma_\beta\big) &\sim \text{Exponential}(1) && \text{[prior for standard deviations]}
\end{aligned}
$$

Why ordinal (`cumulative()`), not categorical/multinomial? The 10 magnitude levels aren't just index labels – they're the literal numeric ordering of `final_score - consensus` ($-0.5 < -0.4 < ... < -0.1 < 0.1 < ... < 0.5$), and treating them as categorical would throw that structure away, treating "+0.3" and "–0.4" as no more related to each other than either is to "+0.1". It also matches the actual data-generating mechanism: this is a continuous quantity (a truncated-normal draw) rounded to one decimal place for recording, not 10 qualitatively distinct outcomes – modeling a discretized continuum with an ordinal model, rather than as unordered categories, is the standard match for that kind of measurement. A multinomial model would also need a full separate set of coefficients (role, expertise, every random effect) for each of the 9 non-reference categories – roughly 9x the parameters of the ordinal specification, most poorly identified given how sparse the tail categories are – versus the ordinal model's single location parameter (and now, a role-specific dispersion parameter) shared across all 10 categories via the threshold structure. That matters substantively too: our actual hypotheses ("panelists' deviations run larger on average," "panelists show more spread") are inherently location/dispersion statements, which only have a natural expression on an ordered scale – a categorical model

has no notion of shift or spread at all, just 10 free-floating probabilities from which those would have to be reconstructed post hoc.

One caveat worth flagging: treating the *full signed* scale as a single ordered dimension is a modeling choice, not the only defensible one. It implicitly assumes direction and magnitude of deviation share one underlying continuum, rather than direction being a separate, qualitatively distinct decision (up vs. down) with magnitude nested inside it – the latter is what the alternative three-part model discussed elsewhere (bernoulli direction + ordinal |magnitude|) would assume instead. We think the single-continuum assumption is the more faithful match to how these scores actually arise (one continuous departure from consensus, not "decide direction, then decide how far"), but it is an assumption.

3 Simulating

For the basic structure of the data-generating process we simulate 50 committees. Rather than fixing the number of applications and members per committee, both now vary to reflect real committee-to-committee heterogeneity: each committee's candidate application pool (before streamlining) is drawn from a beta distribution bounded to CIHR's own stated range of 20-80 applications, and committee size is *derived* from that pool size via a workload target (roughly 5.5 applications reviewed per member) plus lognormal noise, bounded to the real range observed in CIHR's Fall 2025 committee rosters (8-37 members) – reflecting that CIHR sizes committees to manage workload given known application volume, not an independent draw. Each application still gets exactly 3 assigned reviewers regardless of committee size, with the remaining members serving as non-reviewing panelists. Not every candidate application is discussed – CIHR's streamlining rule removes some before discussion (below), leaving roughly 15-16 discussed applications per committee on average, ranging from about 7 to 23 depending on the committee's own pool size and score distribution. A consensus score is drawn per application (committee- and application-level random effects only, no member-level variation yet, since this is before any individual scoring happens). Whether each member deviates from that consensus is a function of their role and self-described expertise; if they deviate, the signed magnitude is drawn from a truncated distribution (its spread now also role-dependent – panelists show a wider range of deviation sizes than reviewers) and rounded to the nearest tenth, matching CIHR's one-decimal-place scoring (with rejection sampling so a "deviated" row can never round down to a contradictory zero).

```
# define parameters
cmte_n = 50          # number of committees

# pool_size (candidate applications per committee, before streamlining)
# varies by committee -- beta-distributed on CIHR's stated [20, 80]
# range -- and mem_n (committee size) is DERIVED from pool_size via a
# workload target, not drawn independently
pool_min = 20
pool_max = 80
target_reviews_per_member = 5.5
mem_min  = 8         # real range, from CIHR's Fall 2025 committee rosters
mem_max  = 37

b0       = 4.0    # intercept for application's true underlying quality
u0c_sd   = 0.1    # random intercept SD for committee (quality level)
```

```
u0a_sd   = 0.3    # random intercept SD for application (quality level)

# signed magnitude of deviation, given deviation occurs, truncated to
# +/- 0.5 (CIHR's stated bound on final vs. consensus score) -- spread
# is now role-dependent, not a single shared value
dev_sd_reviewer = 0.15
dev_sd_panelist = 0.30
dev_min  = -0.5
dev_max  =  0.5

# ... committee/application/member structure (including the pool_size
# -> mem_n derivation), reviewer assignment, and expertise assignment
# omitted here -- see code/ror-sim-deviate.R for the complete script

data <- data |>
  mutate(
    p_dev = plogis(a0 + (a1 * panelist) + (a2 * exp_med) +
      (a3 * exp_low) + (a4 * exp_none)),
    deviated = rbinom(n(), 1, p_dev),
    dev_sd_i = if_else(panelist == 1, dev_sd_panelist, dev_sd_reviewer),
    deviation = if_else(
      deviated == 1,
      round_tenth(rtruncnorm(n(), a = dev_min, b = dev_max,
        mean = dev_bias + u0m_bias, sd = dev_sd_i)),
      0)
  )
```

Every parameter above is at this point just an educated guess and a placeholder, since the point of the simulation is to see whether our modeling approach recovers a known effects, not predictive of what CIHR's actual numbers will look like.

3.1 Simulated data

What did we generate with the parameters above? Figure 2 shows our simulated 50 committees with varying sizes and the number of total and discussed applications. Across the committees the fraction discussed varies from 24.1% to 41.0%. Generally, larger committees end up with more total and more discussed applications.

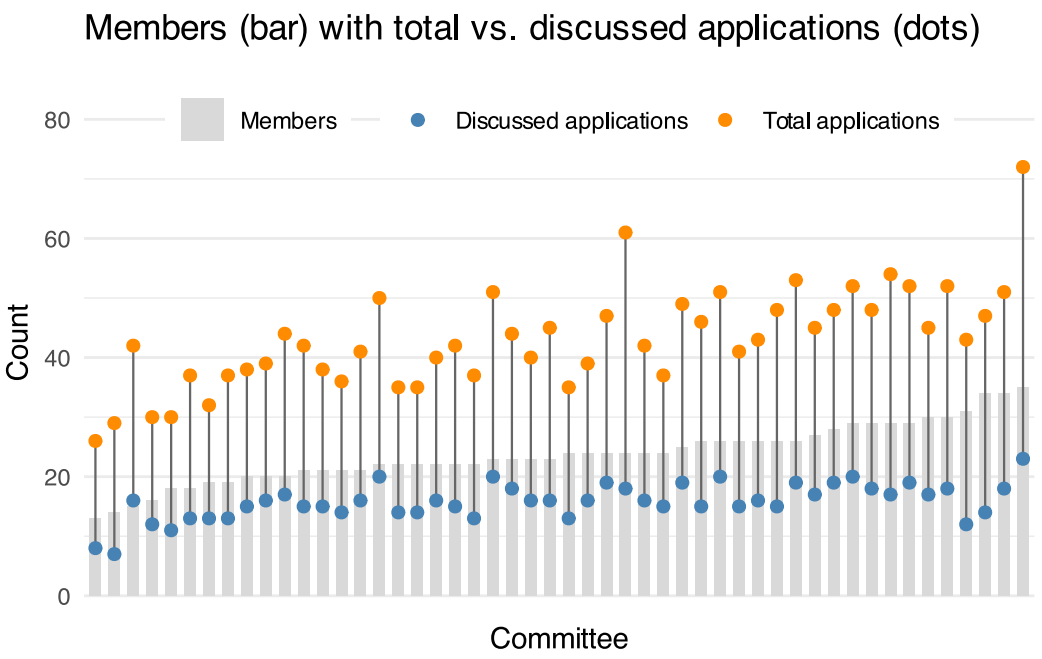

Figure 2: Distribution of committee sizes, total and discussed applications

Figure 3 shows the distribution of initial scores from the three reviewers, the consensus score and then a distribution of overall scores allowing for +/- 0.5 point deviations. Since the likely true consensus scores are not necessarily an average of the 3 reviewer scores (depending on calibration, etc.) we see a small excess in the left tail near 3.5. In practice a consensus score of 3.5 *among discussed applications* is likely more rare, but this seems a reasonable approximation.

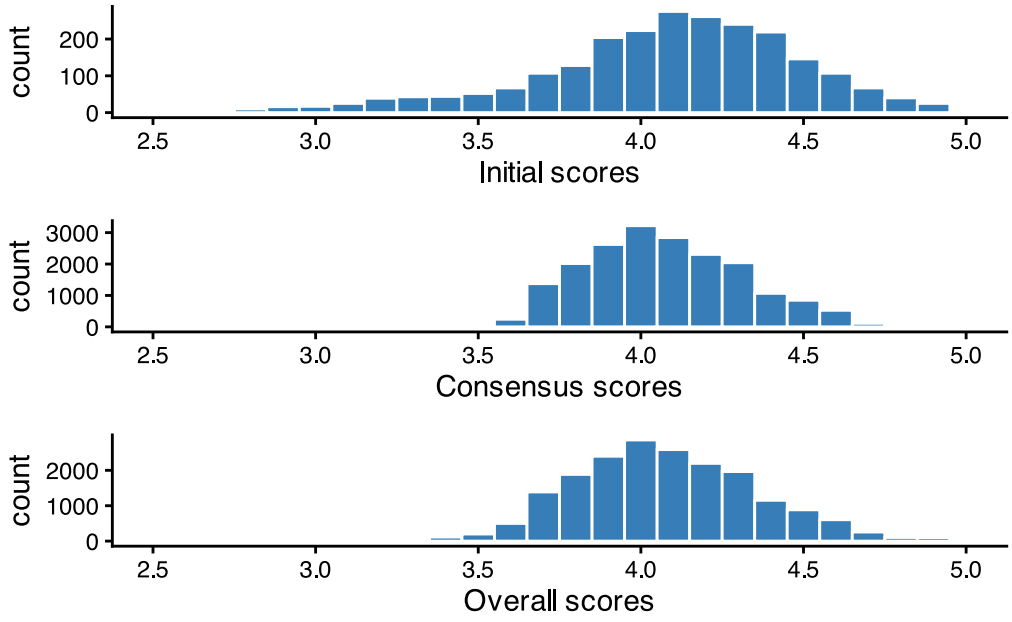

Figure 3: Distribution of simulated intial, consensus, and overall scores

Figure 4 also shows the odd distribution of deviations from the overall consensus score, with a large spike at zero (roughly 2/3rds of committee members going with the consensus):

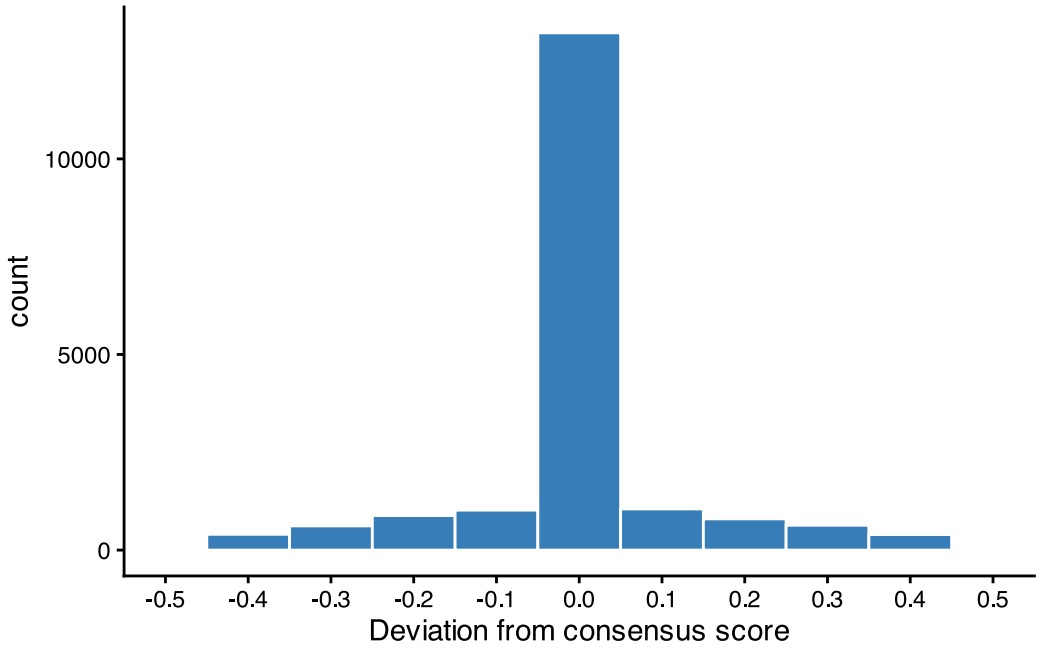

Figure 4: Distribution of deviations from overall consensus scores

Figure 5 shows how our initial simulation incorporates a small degree of variation across committees in the overall scores, as well as the average variation within committees across applications. For the application-level variation we average over committee by within-committee application rank.

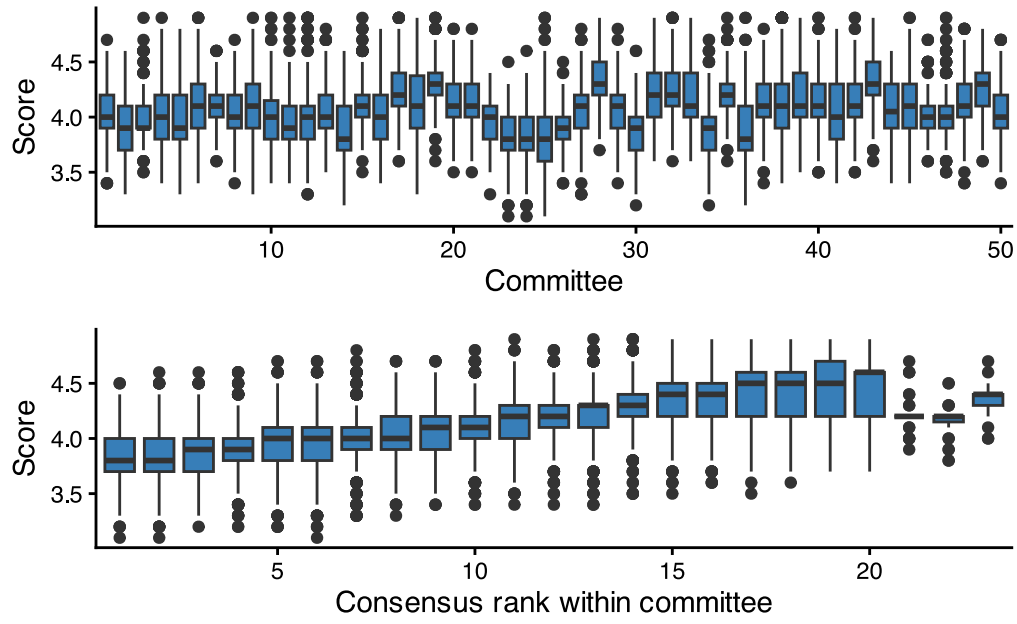

Figure 5: Distribution of score variation by committee and application

3.2 Part 1: did this member deviate?

```
m1_deviate <-
  brm(data = d1,
      family = bernoulli(),
```

```
    deviated ~ 1 + job + exp + (1 | cmte) + (1 | cid) + (1 | app),
    prior = c(prior(normal(0, 1.0), class = Intercept),
              prior(normal(0, 0.5), class = b),
              prior(exponential(1), class = sd)),
    iter = 2000, warmup = 1000, chains = 4, cores = 4,
    sample_prior = "yes",
    control = list(adapt_delta = 0.95))
```

Random intercepts for committee, member (`cid`), and application; fixed effects for role and expertise. Weakly informative priors throughout – `normal(0, 1.5)` on the (logit-scale) intercept, `normal(0, 0.5)` on coefficients, `exponential(1)` on group-level SDs. `sample_prior = "yes"` so we always have prior-predictive draws to check against before trusting the posterior.

This model run on the simulated data gives the following estimates, shown in Table 1:

Table 1

Parameter	Truth	Estimate	Error	95% CrI Lower	95% CrI Upper
Fixed effects (log odds)					
Intercept	−0.5	−0.590	0.062	−0.711	−0.470
Panelist vs. Reviewer	0.3	0.280	0.052	0.180	0.388
Medium vs. High Expertise	−0.3	−0.171	0.061	−0.289	−0.052
Low vs. High Expertise	−0.5	−0.348	0.060	−0.466	−0.229
None vs. High Expertise	−0.8	−0.721	0.062	−0.837	−0.603
Random effects (SD)					
Application		0.043	0.037	0.002	0.127
Committee Member		0.053	0.045	0.003	0.145
Committee		0.018	0.015	0.001	0.058

Estimates from deviation model

From the fixed effects we see that we generally recover the simulated parameters – the 'true' treatment effects in the first column are well approximated by our model. The three random-effect SDs are small and estimated with considerable uncertainty; the simulated data-generating process for whether a member deviates (`deviated`) depends only on `job` and `exp`, with no committee-, application-, or member-level heterogeneity built in at that stage (member-level heterogeneity only enters the *magnitude* of deviation, modeled separately in Part 2 below), so this is the expected pattern. We can also generate the estimated absolute probabilities of deviating and how those are affected by `job` and `expertise` using the `marginaleffects` package.

Table 2

Parameter	P(deviate)	95% CI Lower	95% CI Upper
Overall			
All members	0.317	0.310	0.323
By self-rated expertise			
High	0.416	0.394	0.439
Medium	0.374	0.359	0.390
Low	0.335	0.322	0.347
Not enough	0.257	0.248	0.268
By role			
Reviewer	0.269	0.248	0.287
Panelist	0.324	0.318	0.332

Marginal effects from deviation model

Table 2 show that the overall fraction of members that deviate is around 32%. Reviewers are slightly less likely to deviate than non-reviewing panelists. With respect to expertise, those with high expertise are the least likely to deviate from the consensus (30%), whereas those with no . In practice we

3.3 Part 2: how large, given a deviation?

CIHR scores are entered to one decimal place, so a deviator's score can only land on one of 10 discrete steps ($\pm 0.1 \ldots \pm 0.5$ relative to consensus). That's genuinely ordinal/discrete data, not a continuous quantity with occasional extreme values – a continuous family (`student()`/`gaussian()`) would assign probability to impossible values and has no natural bound at ± 0.5 without truncation hacks. We use an ordinal `cumulative()` model instead, with flexible (non-equidistant) thresholds, since there's no reason to assume the 10 steps are equally likely – and in the simulated data they clearly aren't (concentrated near ± 0.1, sparse at ± 0.5, which happen to be the swings most relevant to Aim 3's funding-decision question).

```
m1_magnitude <-
  brm(data = d1_dev,
      family = cumulative(link = "logit", threshold = "flexible"),
      deviation ~ 1 + job + exp + (1 | cmte) + (1 | cid) + (1 | app),
      prior = c(prior(normal(0, 1.5), class = Intercept),
                prior(normal(0, 0.5), class = b),
                prior(exponential(1), class = sd)),
      iter = 2000, warmup = 1000, chains = 4, cores = 4,
      sample_prior = "yes",
      control = list(adapt_delta = 0.95))
```

The cost: $E[\text{magnitude} \mid \text{deviate}]$ from an ordinal model is not a linear prediction – it's a probability-weighted sum over the 10 category values, computed per posterior draw, not read off a default `marginaleffects` contrast. That combination step (and the corresponding $E\left[d_{ijk}\right] = P(\text{deviate}) \times E[\text{magnitude} \mid \text{deviate}]$ calculation across both models) is written but not yet implemented in code – see Open Questions below.

Table 3

Parameter	Truth	Estimate	Error	95% CrI Lower	95% CrI Upper
Thresholds					
Threshold 1 (-0.5 | −0.4)		−3.327	0.598	−4.609	−2.303
Threshold 2 (-0.4 | −0.3)		−2.046	0.372	−2.833	−1.406
Threshold 3 (-0.3 | −0.2)		−1.263	0.232	−1.768	−0.872
Threshold 4 (-0.2 | −0.1)		−0.595	0.120	−0.864	−0.396
Threshold 5 (-0.1 | 0.1)		0.010	0.055	−0.101	0.121
Threshold 6 (0.1 | 0.2)		0.640	0.125	0.424	0.913
Threshold 7 (0.2 | 0.3)		1.259	0.228	0.859	1.740
Threshold 8 (0.3 | 0.4)		2.078	0.367	1.428	2.843
Threshold 9 (0.4 | 0.5)		3.486	0.619	2.390	4.782
Fixed effects, location (log-cumulative-odds)					
Panelist vs. Reviewer (location)	0	0.012	0.048	−0.082	0.110
Medium vs. High Expertise	0	−0.008	0.056	−0.127	0.106
Low vs. High Expertise	0	−0.054	0.063	−0.182	0.066
None vs. High Expertise	0	−0.024	0.062	−0.155	0.101
Random effects (SD)					
Application	0	0.044	0.038	0.002	0.147
Committee Member		0.401	0.079	0.269	0.579
Committee	0	0.068	0.043	0.005	0.159
Fixed effects, dispersion (log-scale)					
Reviewer (dispersion)		0.650	0.179	0.324	1.022
Panelist (dispersion)		0.129	0.179	−0.193	0.496

Estimates from magnitude model

Below you can see the average marginal predictions for reviewers vs. panelists in terms of deviations from the consensus score, given that a deviation occurred:

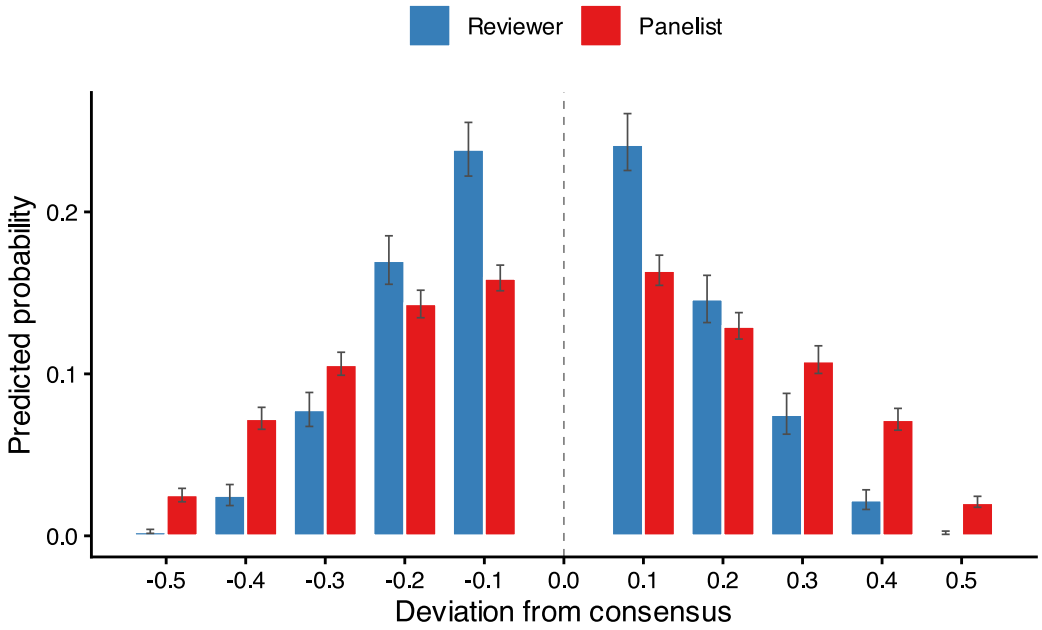

Figure 6: Predicted magnitude of deviation, by role

These differences in predicted probabilities for deviations of different magnitudes can also be expressed as marginal effects in Figure 7:

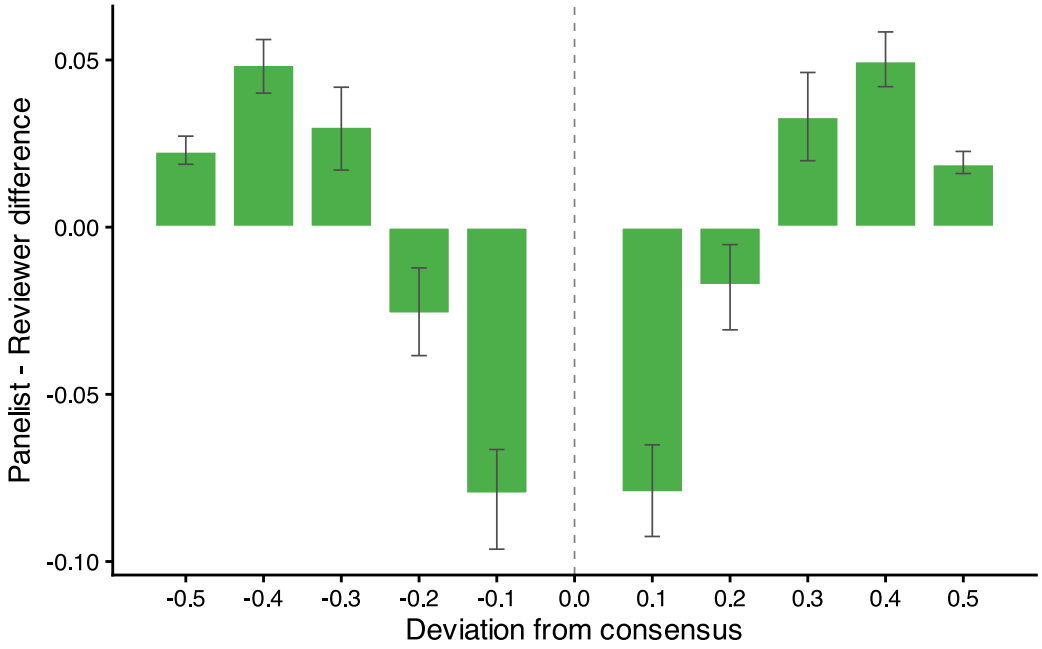

Figure 7: Marginal effects of job on the magnitude of deviations

3.4 Software

Bayesian throughout, via `brms` + `cmdstanr` (Stan backend), matching the conventions in our other multilevel work. `marginaleffects` for posterior contrasts; models cached to `code/fits/` and deliberately re-fit (not silently reused) whenever formula or data change.

4 Extending to Aim 2: applicant characteristics

Since applicant gender and career stage will never be extractable – not even as CIHR's own dummy data – this simulation is the only pre-deployment test that code touching those variables gets. We added them as **application-level** attributes (one value per application, like consensus score, not per member) to a new, separate simulation script (`code/ror-sim-aim2.R`) rather than growing the Aim 1 script further – see "A note on simulation architecture" below.

Critically, we didn't just add *main* effects of gender/career stage. Aim 2's actual question is whether the *engagement/expertise effect on deviation* differs by applicant characteristic – which requires an **interaction** term to exist in the generating process, or there's nothing for the fitted model to recover.

```
# applicant-level main effects (modest -- not the effects of interest)
g1       =  0.1   # applicant gender = female
c1       = -0.1   # applicant career stage = early

# Aim 2's actual target: does the reviewer-engagement / expertise effect
# on P(deviate) differ by applicant gender / career stage? Baked in large
# on purpose so we can confirm the two-part model machinery actually
# recovers a real interaction, not just a real main effect.
i_panelist_female     =  0.5  # job(panelist) x gender(female)
i_expnone_earlycareer =  0.6  # exp(none) x career_stage(early)

data <- data |>
  mutate(
    p_dev = plogis(a0 + (a1 * panelist) + (a2 * exp_med) +
      (a3 * exp_low) + (a4 * exp_none) +
      (g1 * female) + (c1 * early_career) +
      (i_panelist_female * panelist * female) +
      (i_expnone_earlycareer * exp_none * early_career)),
    deviated = rbinom(n(), 1, p_dev)
    # ... deviation magnitude drawn the same way as Aim 1; applicant
    # terms are deliberately left out of the magnitude part, consistent
    # with the Aim 1 finding above that signal lives in whether someone
    # deviates, not in the average size of the deviation
  )
```

To be clear about what these numbers are and aren't: they are chosen to be large enough to detect cleanly in a simulation of this size, not estimates – or even guesses – about the true magnitude or even direction of any real effect in CIHR review. The goal is purely to confirm our statistical machinery would notice an interaction if the real data contained one.

```
d2 <- read_csv(here("data", "sim-aim2-data.csv"), show_col_types = FALSE)

m_aim2 <- glm(deviated ~ job * gender + exp * career_stage,
  data = d2, family = binomial())

coefs <- summary(m_aim2)$coefficients
coefs[grepl(":", rownames(coefs)), ] |>
  as_tibble(rownames = "term") |>
  tt(digits = 3)
```

term	Estimate	Std. Error	z value	Pr(>|z|)
jobreviewer:gendermale	0.571	0.0933	6.12	0.000000000933
explow:career_stageestablished	0.26	0.1228	2.12	0.034154693615
expmed:career_stageestablished	0.253	0.1281	1.97	0.048458324937
expnone:career_stageestablished	−0.43	0.1195	−3.6	0.00032331474

Both interactions are recovered cleanly and with high significance (the sign flips relative to the parameters above are just R's alphabetical choice of reference category – `job`'s reference is "panelist", `gender`'s is "female", `career_stage`'s is "early", so e.g. `jobreviewer:gendermale` is algebraically the same interaction as `job(panelist):gender(female)`, just from the opposite corner of the 2x2).

Not yet built: Aim-2-specific versions of `code/ror-analysis-deviate-model.R`/`ror-analysis-magnitude-model.R` that actually include these interaction terms in the `brm()` formulas. The simulation is ready; the model scripts that would be tested against it aren't written yet.

5 Aim 3: not yet designed

Aim 3 asks how alternative funding-decision schemes – reweighting scores by engagement, or partially randomizing funding decisions for applications near the threshold – would compare to the status quo. That's a structurally different kind of simulation: an intervention/counterfactual layered on top of the funding decision itself, not just an extension of Aims 1-2′s data-generating process for reviewer behavior. We haven't started designing it, and would welcome input on what the alternative schemes worth simulating actually are before building anything.

Structurally, nothing in the current pipeline goes past `score` (the final, post-discussion score) – there's no funding-decision layer at all yet (rank by final score, allocate against a budget, apply whatever equalization overlay CIHR actually uses). Aim 3 can't be simulated without adding one. This is also where the partial-randomization idea specifically earns its keep: informally, applications near the funding threshold can end up clustered close enough together that discussion- driven movement alone is plausibly enough to reorder who clears the line, which is exactly the kind of uncertainty a modified-lottery scheme is designed to acknowledge rather than paper over with strict rank order. Worth real data once it arrives from CIHR to confirm how tight that clustering actually is.

6 A note on simulation architecture

We're deliberately keeping a separate, self-contained simulation script per aim (`ror-sim-deviation.R` for Aim 1, `ror-sim-aim2.R` for Aim 2, and eventually an Aim 3 script) rather than one script that keeps growing. Some duplication of the committee/application/member setup across scripts is an accepted cost of each one being independently runnable and easy to reason about in isolation.

7 What this simulation validates, and what it doesn't

It validates that: (1) the two-part model structure is necessary, not over-engineering, for this outcome; (2) the ordinal magnitude submodel is the right match for CIHR's one-decimal-place scoring; (3) the model can recover both main effects (Aim 1) and interaction effects (Aim 2) of a detectable size, using only the fields CIHR has confirmed are extractable plus the fields that would only ever be touched via an in-house CIHR-run analysis.

It does **not** tell us anything about the true size, direction, or even existence of any of these effects in actual CIHR peer review – every parameter in both simulations is an illustrative placeholder chosen for statistical detectability, not drawn from evidence.

8 Open questions

8.1 For Arijit

- Is d_{ijk} (final − consensus) the right primary outcome, or is there a better way to frame the estimand for Aim 1/2?
- Does the interaction specification for Aim 2 (`job x gender`, `exp x career_stage`, both entering only the deviation-probability part) match how you'd operationalize the research question, or would you frame it differently?
- Any concerns about the priors, the random-effects structure, or the choice of an ordinal rather than continuous family for the magnitude part?
- Anything about the overall statistical strategy that looks wrong before this goes to CIHR?

8.2 For CIHR's Funding Analytics Team

(Carried over from `code/ror-research-log.qmd`; repeated here since this document is the more likely vehicle for actually sending them.)

1. For fields that can't be extracted (applicant gender/career stage, reviewer gender/experience, etc.) – can the dummy-data step still include *synthetic* versions of those columns for us to pipeline-test against, or is the in-house-run path a black box until the real run?
2. Resubmission status is only available from 2023 onward – does that mean our cohort has to start at the 2023 competition if we want that field, or can it be requested as a partially-missing covariate over a longer window?
3. How should we define/measure "reviewer experience" for the fields where that's the blocker to a yes/no on extractability?
4. Can you confirm our understanding of the review process itself (the ± 0.5 rule, one-decimal-place scoring, equal-weight averaging into the funding decision) is accurate?
5. How much variation exists in the number of members across committees, applications per committee, and scores per application (given conflicts)?
6. Discussion time? Do we know how long the discussion period goes at the application level?

9 What comes next

Once the Aim 2 model script exists and `FIT_MODELS` is switched on – after confirming CIHR's execution environment can actually compile/run Stan via `cmdstanr`, and after prior-predictive checks on both submodels – this document (or its next revision) will add a results section presenting posterior marginal effects via `marginaleffects` for both the deviation-probability and magnitude submodels, and showing recovery of the known simulated effects at the full posterior level, the same way the frequentist checks above do at a point-estimate level.

10 Appendix: Aim 1 simulation script (`ror-sim-deviate.R`)

The "Model specification" section above walks through a simplified, didactic version of the Aim 1 data-generating process. `code/ror-sim-deviate.R` is the actual, current simulation script – the one that produced the results reported in that section and the workhorse we're using going forward – and

it's more procedurally involved than the summary above lets on. Rather than re-typing or excerpting pieces of it here (a second, hand-maintained copy that can silently drift out of sync with the real script, a failure mode we've already hit once with the simulation's own file names), this appendix pulls in the script's full, current contents directly, so it can never go stale. What follows is a guide to what to look for as you read it.

The script is organized into seven numbered sections, matching the `## N ...` comments in the source:

- **0-1: packages and parameters.** All parameters are illustrative placeholders, not estimates – with two exceptions called out explicitly in the script's own comments: `streamline_rank_threshold = 0.60` (CIHR's stated real streamlining rule) and the beta-distributed committee pool size range, `[20, 80]` (CIHR's own stated range). `init_sd_lo` and the `exp` category mix were calibrated against real reviewer-score data Sam has access to, though that source data itself isn't reproduced or logged anywhere in this repository.
- **2: candidate pool.** Generates each committee's full pool of candidate applications (before streamlining), assigns 3 of each application's 24 committee members as reviewers, draws each reviewer's initial score, and has each reviewer separately record a "top"/"bottom" call – correlated with, but not determined by, their score. Out-of-bounds initial scores are redrawn rather than clamped, to avoid an artificial probability spike at the scale's boundaries. The application grid is built manually (not via `faux::add_random()`) specifically to avoid a conflation bug found earlier in this project: `add_random()`'s crossed factors give identical random-effect draws to same-labeled applications across different committees unless a unique composite key (`aid`) is used instead.
- **3: streamlining decision.** Implements CIHR's actual rule as Sam described it from his own committee/Scientific Officer experience: an application is streamlined out (not discussed) if at least one reviewer called it "bottom" *and* its mean-of-3 score ranks in the bottom 60% of that committee's own candidate pool – a relative, within-committee rule, not a fixed absolute score cutoff. This mechanism was specifically validated against two hypothetical datasets Sam constructed, including a case where two applications with identical reviewer scores had different discussed/not-discussed outcomes – something a pure score-threshold rule can't produce, but this rank-and-call combination can. An earlier "bring back" stage, where a reviewer could argue to reinstate a borderline streamlined-out application, was tried and deliberately dropped as too haphazard and too thinly grounded (only ~6% of discussed applications) to be worth the added complexity – see `code/ror-research-log.qmd` for that discussion.
- **4: the two-part deviation model.** Once discussed, each member's score is generated as consensus plus a two-part process: whether they deviate from consensus at all (a function of role and self-described expertise), and, if so, the signed magnitude (a truncated, rounded draw, with member-level leniency/harshness heterogeneity, `u0m_bias`). This is the same two-part structure the modeling strategy above is built to recover.
- **5: internal consistency checks.** A set of `stopifnot()` assertions – e.g., every committee retains at least one discussed application, consensus never falls outside the range of its own 3 reviewer scores, deviation is exactly zero whenever `deviated == 0` and never zero when `deviated == 1`. These aren't diagnostics for a reader; they're guardrails that halt the script if a future edit breaks an invariant the rest of the design depends on.
- **6: empirical checks.** Printed summaries – pool size and discussion-rate distributions, confirmation that the identical-scores-different-outcomes pattern the mechanism was designed to explain actually occurs, and a quick `glm()`/`lmer()` check that `job`/`exp` effects on deviation and member-level heterogeneity both still recover cleanly despite the richer selection stage.

- **7: output.** Writes `data/sim-deviate.csv`, which `code/ror-analysis-deviate-model.R` and `code/ror-analysis-magnitude-model.R` read.

```
#  program:  ror-sim-deviate.R
#  task:     generating data for Aim 1
#  input:    none (simulated from scratch)
#  output:   data/sim-deviate.csv
#  project:  RoR
#  author:   sam harper \ 2026-08-20
#
#  note:       1. each of the 3 assigned reviewers gives a score AND a
#                 separate categorical "top" (competitive) / "bottom"
#                 (not competitive) call -- correlated with their score,
#                 but not a deterministic function of it.
#              2. streamlining rule: an application is streamlined out
#                 (not discussed) iff >=1 reviewer called it "bottom"
#                 AND its mean-of-3 score ranks in the bottom 60% of
#                 *that committee's own* candidate pool (a relative/rank
#                 rule, not a fixed absolute score).


##  0 Load needed packages ----
library(here)
library(tidyverse)
library(faux)
library(truncnorm)

set.seed(20260819)

##  1 Define parameters ----

cmte_n = 50     # number of committees

# Per-committee pool_size is drawn (below, section 2) from a beta
# distribution scaled to CIHR's own stated 20-80 range. mem_n is then
# DERIVED from pool_size.  Noise (lognormal,
# below) keeps real committee-to-committee variation rather than
# forcing a fixed ratio.

pool_min = 20        # lower bound on applications reviewed per committee
pool_max = 80        # upper bound
pool_shape1 = 2.5    # rbeta() shape -- illustrative, not fit; chosen to
pool_shape2 = 4      # roughly match the mean/SD/right-skew of the
                     # funded-count back-calculation (mean ~42, SD ~12)

mem_min = 8          # lower bound (from CIHR's Fall 2025 Proj Grant
mem_max = 37         # committee roster, cihr-irsc.gc.ca/e/54732.html)

# workload target: roughly 5 applications reviewed per member
target_reviews_per_member = 5.5
mem_noise_sd = 0.15   # lognormal noise SD (log scale) around the
                      # workload

b0         = 4.0    # intercept for (discussed) application's true quality
```

```
u0c_sd     = 0.1    # random intercept SD for committee (quality level)
u0a_sd     = 0.3    # random intercept SD for application (quality level)

# longer tail for lower-ranked applications.
# streamlining rule (section 3) means a reviewer's
# very low individual score can only survive to discussion if the
# other 2 reviewers' scores are high enough to keep the consensus above
# the committee's 60th-percentile rank
init_sd_lo = 1.1
init_sd_hi = 0.15   # reviewer-noise SD above it (tighter)

# probability of deviation from consensus (logit scale), and the
# signed magnitude given deviation
# this is the "hurdle"-equivalent part
# reference category is HIGH expertise (2026-08-20, switched from
# medium for simplicity) -- re-derived to reproduce the exact same
# per-category probabilities as before, just relabeled: a0 is now the
# high-expertise/reviewer baseline logit, a2-a4 are med/low/none
# relative to high instead of high/low/none relative to med.
a0       = -0.5   # baseline logit prob. of deviating (high expertise, reviewer)
a1       =  0.3   # panelist vs. reviewer
a2       = -0.3   # medium expertise (vs. high)
a3       = -0.5   # low expertise (vs. high)
a4       = -0.8   # no expertise (vs. high)

# signed magnitude of deviation, given deviation occurs, truncated to
# +/- 0.5 (CIHR's stated bound on final vs. consensus score)
dev_bias = 0       # population-average bias: still none (see u0m_bias_sd
# below for between-member variation around this)
# magnitude SD differs by job: panelists swing harder conditional on
# deviating (heavier tail toward +/-0.5), not just more often (that's
# the deviation-probability model above, a0-a4) -- 2026-08-25, tuned
# empirically (see research log) so P(|deviation|>=0.3) is ~3x higher
# for panelists than reviewers, since the single 0.2 category itself
# saturates well under 16% for any sd (mass beyond that point keeps
# shifting further out to 0.3-0.5 instead of piling up at 0.2)
dev_sd_reviewer = 0.15
dev_sd_panelist = 0.30
dev_min  = -0.5
dev_max  =  0.5

# between-member SD in habitual leniency/harshness -- some members
# consistently deviate a bit high, some a bit low, across every
# application they review, but the population average stays at
# dev_bias (0). Not a fixed direction for everyone (that would just be
# dev_bias != 0); this is heterogeneity *across* members. One draw per
# unique cid, not per raw member-slot.
u0m_bias_sd = 0.1

scale_min = 0     # lower bound of the scoring scale
score_max = 4.9   # upper bound of the scoring scale

round_tenth <- function(x) round(x * 10) / 10
```

```r
# -- streamlining mechanism parameters --

# each reviewer's "top"/"bottom" call: p(bottom) = plogis(tb_slope *
# (tb_center - score)) -- tb_center is roughly where a reviewer is as
# likely to say "top" as "bottom"; tb_slope controls how tightly the
# call tracks their own score (higher = closer to deterministic)
tb_center = 3.9
tb_slope  = 6

# initial rule: streamlined out iff >=1 "bottom" call AND rank (of the
# mean of 3 scores, within this committee's own candidate pool) is at
# or below this percentile.
streamline_rank_threshold = 0.60

##  2 Stage 1: candidate pool -- reviewer scores and top/bottom calls ----
## Mirrors ror-sim-aim1.R's committee/application/member setup and
## reviewer-score generation exactly (including the redraw-not-clamp
## handling of out-of-bounds scores); the streamlining logic that
## follows in section 3, and the variable per-committee pool_size below,
## are new.
##
## The grid is built manually (reframe + cross_join) rather than via
## add_random(application = ...), because add_random()'s "application"
## factor would be fully crossed with committee (same pool_per_cmte
## levels recycled identically in every committee) even when pool sizes
## vary by committee -- and add_ranef() on that raw factor would give
## identical u0a draws to same-labeled applications across different
## committees. This is the same conflation bug found and fixed in
## ror-sim-aim1.R/ror-sim-aim2.R on 2026-08-14 (there for "member";
## fixed here by never creating the crossed factor in the first place
## for "application"), via an explicit unique `aid` composite key.

cmte_pool <- tibble(
  cmte = sprintf("%02d", 1:cmte_n),
  pool_size = round(pool_min +
    (pool_max - pool_min) * rbeta(cmte_n, pool_shape1, pool_shape2))
) |>
  mutate(mem_n_center = 3 * pool_size / target_reviews_per_member)

# mem_n derived from pool_size (workload-target center) plus lognormal
# noise, redrawn (not clamped) if it lands outside the real observed
# [mem_min, mem_max] range -- redrawing preserves the noise
# distribution's shape near the boundary instead of piling probability
# up at a clamped edge (same rejection-sampling pattern used throughout
# this script for other bounded quantities).
cmte_pool$mem_n <- round(cmte_pool$mem_n_center *
  exp(rnorm(cmte_n, 0, mem_noise_sd)))
out_idx <- which(cmte_pool$mem_n < mem_min | cmte_pool$mem_n > mem_max)
while (length(out_idx) > 0) {
  cmte_pool$mem_n[out_idx] <- round(cmte_pool$mem_n_center[out_idx] *
    exp(rnorm(length(out_idx), 0, mem_noise_sd)))
  out_idx <- which(cmte_pool$mem_n < mem_min | cmte_pool$mem_n > mem_max)
}
cmte_pool <- cmte_pool |> select(-mem_n_center)
```

```r
cmte_app <- cmte_pool |>
  reframe(app = seq_len(pool_size), .by = c(cmte, pool_size)) |>
  select(cmte, app)

cmte_mem <- cmte_pool |>
  reframe(memno = sprintf("%02d", seq_len(mem_n)), .by = c(cmte, mem_n)) |>
  select(cmte, memno)

# reviewer-assignment weights by self-rated expertise
# real committees don't assign the 3 reviewers independently of
# expertise (expertise strongly increases reviewer odds);
# Used directly below as sample() selection weights (not
# where high are 40x more likely than none, etc.
exp_reviewer_weight <- c(high = 40, med = 30, low = 5, none = 1)

# cmte_app x cmte_mem joined by "cmte" only (not a blanket cross_join)
# so each committee only crosses with its OWN members -- mem_n now
# varies by committee, so a single global memno range no longer applies
# to every committee the way it did when mem_n was fixed
data <- cmte_app |>
  left_join(cmte_mem, by = "cmte", relationship = "many-to-many") |>
  mutate(
    cid = paste0(cmte, "_", memno),
    aid = paste0(cmte, "_", app)
  ) |>

  add_ranef("cmte", u0c = u0c_sd) |>
  add_ranef("aid", u0a = u0a_sd) |>

  group_by(cmte, app) |>
  mutate(
    # self-rated expertise mix -- weighted draw of size n() (was a
    # fixed-count permutation of exactly 24, which only worked when
    # every committee had exactly 24 members; a weighted sample with
    # replacement generalizes to the now-variable group size, at the
    # cost of realized per-application counts varying stochastically
    # around these proportions rather than landing on them exactly)
    exp = sample(c("high", "med", "low", "none"), size = n(),
      replace = TRUE, prob = c(high = 2, med = 5, low = 7, none = 10)),
    # reviewer assignment a weighted draw per above
    job = {
      reviewer_pos <- sample.int(n(), size = 3, replace = FALSE,
        prob = exp_reviewer_weight[exp])
      if_else(seq_len(n()) %in% reviewer_pos, "reviewer", "panelist")
    },
    z_init = rnorm(n()),
    init_score = if_else(job == "reviewer",
      round_tenth(b0 + u0c + u0a +
        if_else(z_init < 0, z_init * init_sd_lo, z_init * init_sd_hi)),
      NA_real_)
  ) |>
  ungroup() |>
  select(-z_init)
```

```
# redraw any reviewer's init_score outside the scale's true bounds
out_idx <- which(!is.na(data$init_score) &
  (data$init_score < scale_min | data$init_score > score_max))
while (length(out_idx) > 0) {
  z <- rnorm(length(out_idx))
  noise <- if_else(z < 0, z * init_sd_lo, z * init_sd_hi)
  data$init_score[out_idx] <- round_tenth(
    b0 + data$u0c[out_idx] + data$u0a[out_idx] + noise)
  out_idx <- which(!is.na(data$init_score) &
    (data$init_score < scale_min | data$init_score > score_max))
}

# each reviewer's separate top/bottom call
data <- data |>
  mutate(
    p_bottom = plogis(tb_slope * (tb_center - init_score)),
    # rbinom() on the NA p_bottom values (panelist rows) warns even
    # though the result is discarded by the outer if_else() below --
    # substitute a dummy 0 there rather than let it warn every run
    top_bottom = if_else(job == "reviewer",
      if_else(rbinom(n(), 1, if_else(is.na(p_bottom), 0, p_bottom)) == 1,
        "bottom", "top"),
      NA_character_)
  ) |>
  select(-p_bottom)

##  3 Streamlining decision ----

# application-level summaries, broadcast back to every member-row
data <- data |>
  group_by(cmte, app) |>
  mutate(
    consensus = round_tenth(mean(init_score, na.rm = TRUE)),
    any_bottom = any(top_bottom == "bottom", na.rm = TRUE)
  ) |>
  ungroup()

# decide streamlining once per application (on distinct apps),
# rank has to be computed against other *applications*, not other rows)
# then join back
decision <- data |>
  distinct(cmte, app, consensus, any_bottom) |>
  group_by(cmte) |>
  mutate(rank_pct = percent_rank(consensus)) |>
  ungroup() |>
  mutate(
    discussed = !(any_bottom & (rank_pct <= streamline_rank_threshold))
  ) |>
  select(cmte, app, consensus, rank_pct, discussed)

stopifnot(
  "every committee should have at least 1 discussed application" =
    data |> left_join(decision |> select(-consensus), by = c("cmte", "app")) |>
```

```
      filter(discussed) |> distinct(cmte, app) |>
      count(cmte) |> pull(n) |> min() >= 1
)

data <- data |>
  left_join(decision |> select(-consensus), by = c("cmte", "app")) |>
  filter(discussed) |>

  mutate(
    panelist = if_else(job == "panelist", 1, 0),
    exp_med  = if_else(exp == "med", 1, 0),
    exp_low  = if_else(exp == "low", 1, 0),
    exp_none = if_else(exp == "none", 1, 0)
  ) |>

  # member-level leniency/harshness trait
  # add_ranef on cid
  add_ranef("cid", u0m_bias = u0m_bias_sd)

##  4 Two-part deviation from consensus ----
## CIHR scores (both consensus and individual final scores) can only be
## entered to one decimal place, so deviation -- and hence score -- must
## also land on a tenth. A raw truncnorm() draw is continuous, so we round
## it and then redraw any deviated == 1 case that rounds to exactly 0 (a
## "deviator" can't end up with a final score identical to consensus).

data <- data |>
  mutate(
    p_dev = plogis(a0 + (a1 * panelist) + (a2 * exp_med) +
      (a3 * exp_low) + (a4 * exp_none)),
    deviated = rbinom(n(), 1, p_dev),
    dev_sd_i = if_else(panelist == 1, dev_sd_panelist, dev_sd_reviewer),
    deviation = if_else(
      deviated == 1,
      round_tenth(rtruncnorm(n(), a = dev_min, b = dev_max,
        mean = dev_bias + u0m_bias, sd = dev_sd_i)),
      0)
  )

zero_idx <- which(data$deviated == 1 & data$deviation == 0)
while (length(zero_idx) > 0) {
  data$deviation[zero_idx] <- round_tenth(
    rtruncnorm(length(zero_idx), a = dev_min, b = dev_max,
      mean = dev_bias + data$u0m_bias[zero_idx], sd = data$dev_sd_i[zero_idx]))
  zero_idx <- which(data$deviated == 1 & data$deviation == 0)
}

data <- data |>
  mutate(
    score = round_tenth(pmax(scale_min,
      pmin(score_max, consensus + deviation)))
  ) |>
  select(-u0c, -u0a, -u0m_bias, -p_dev)
```

```r
##  5 Checks ----

stopifnot(
  "expect exactly 3 reviewers per committee-application" =
    data |> filter(job == "reviewer") |>
      count(cmte, app) |> pull(n) |> unique() == 3,
  "every application within a committee should have the same member count as that
committee's own mem_n" =
    data |> count(cmte, app) |>
      left_join(cmte_pool |> select(cmte, mem_n), by = "cmte") |>
      summarise(ok = all(n == mem_n)) |> pull(ok),
  "expect exactly 3 non-missing initial reviewer scores per application" =
    data |> filter(!is.na(init_score)) |>
      count(cmte, app) |> pull(n) |> unique() == 3,
  "consensus should never fall outside the range of the 3 reviewers' own initial scores" =
    data |> group_by(cmte, app) |>
      summarise(
        lo = min(init_score, na.rm = TRUE),
        hi = max(init_score, na.rm = TRUE),
        cons = first(consensus), .groups = "drop") |>
      summarise(ok = all(cons >= lo & cons <= hi)) |> pull(ok),
  "every retained application should actually be flagged discussed" =
    all(data$discussed),
  "deviated should be 0/1" =
    all(data$deviated %in% c(0, 1)),
  "deviation should be exactly 0 when deviated == 0" =
    all(data$deviation[data$deviated == 0] == 0),
  "score should stay within [scale_min, score_max]" =
    all(data$score >= scale_min & data$score <= score_max),
  "deviated == 1 rows should never have a zero deviation after rounding" =
    all(data$deviation[data$deviated == 1] != 0)
)

##  6 Empirical checks ----

n_candidates_total <- sum(cmte_pool$pool_size)
n_discussed_total <- data |> distinct(cmte, app) |> nrow()

cat("=== per-committee pool size (drawn from beta on CIHR's stated [20,80] range) ===\n")
print(summary(cmte_pool$pool_size))
cat("SD across committees:", round(sd(cmte_pool$pool_size), 2), "\n\n")

cat("=== per-committee member count (drawn from beta on real Fall 2025 roster range) ===\n")
print(summary(cmte_pool$mem_n))
cat("SD across committees:", round(sd(cmte_pool$mem_n), 2), "\n\n")

cat("=== streamlining mechanism ===\n")
cat("overall discussion rate:",
  round(n_discussed_total / n_candidates_total, 3), "\n")

per_cmte <- data |> distinct(cmte, app) |> count(cmte, name = "n_discussed")
cat("discussed applications per committee -- summary:\n")
print(summary(per_cmte$n_discussed))
cat("SD across committees:", round(sd(per_cmte$n_discussed), 2), "\n\n")
```

```
cat("=== can identical consensus scores still produce different outcomes? ===\n")
## uses `decision`, which covers the FULL candidate pool (both discussed
## and streamlined-out) -- `data` by this point only contains the
## discussed survivors, so checking against `data` alone could never
## find a not-discussed match
dupe_check <- decision |>
  group_by(consensus) |>
  filter(n_distinct(discussed) > 1) |>
  ungroup()
cat("distinct consensus values with both outcomes present:",
  n_distinct(dupe_check$consensus), "\n")
dupe_check |> arrange(consensus) |>
  select(cmte, app, consensus, rank_pct, discussed) |>
  slice_head(n = 8) |> print()
cat("\n")

cat("=== does job/exp on P(deviate) still recover cleanly despite the richer selection
stage? ===\n")
m_check <- glm(deviated ~ job + exp, data = data, family = binomial())
print(summary(m_check)$coefficients)

cat("\n=== member-level heterogeneity still recoverable? ===\n")
suppressMessages(library(lme4))
m_member_check <- lmer(deviation ~ 1 + (1 | cid),
  data = data |> filter(deviated == 1))
print(VarCorr(m_member_check))

##  7 Write output ----

write_csv(data, here("data", "sim-deviate.csv"))

# cmte_pool (pool_size/mem_n before streamlining) saved separately --
# data/sim-deviate.csv only has discussed applications, so this is the
# only place pool_size/mem_n survive. Consumed by
# writing/ror-modeling-strategy.qmd's fig-cmte, so it stays consistent
# with whatever data/sim-deviate.csv this same run produced, instead of
# being hand-recreated there.
dir.create(here("output"), showWarnings = FALSE, recursive = TRUE)
saveRDS(cmte_pool, here("output", "cmte-pool.rds"))
```

11 References

Mullahy, John. 1986. “Specification and Testing of Some Modified Count Data Models.” *Journal of Econometrics* 33 (3): 341365.